Ed-Feed Sentiment Analysis

Dataset Overview, Baseline Model, Advanced Model, and TensorFlow Model

1. Dataset

1.1 Overview

The dataset used across all three models is the EduFeed dataset, a collection of student feedback comments about professors sourced from a rate-my-professor style platform. The balanced version of this dataset, named balanced_edufeed_dataset.xlsx, contains 35,421 rows and 62 columns. The dataset has been preprocessed and balanced so that each of the three sentiment classes, Positive, Neutral, and Negative, contains exactly 11,807 records, ensuring no class dominates model training.

1.2 Key Columns

The dataset contains a rich set of features spanning multiple categories. The professor and institution identifiers include professor_name, school_name, department_name, local_name, and state_name. The rating fields include star_rating (range 1.05 to 5.0, mean 3.48), student_star, student_difficult, and diff_index. The text field comments holds the raw student feedback text, which is the primary input to all models. The post_date, post_year, and post_month fields capture when the review was submitted. The year_since_first_review field ranges from 0 to 16 with a mean of approximately 7.78 years. Behavioral tags such as gives_good_feedback, caring, respected, tough_grader, lots_of_homework, and accessible_outside_class provide additional context about professor attributes. Derived features include tag_count (mean 3.79, max 19), help_ratio, difficulty_category, experience_bucket, comment_length_cat, grade_group, and course_mode. The target column is sentiment_label, which takes the values Positive, Neutral, or Negative.

1.3 Dataset Statistics

Total records: 35,421

Total features: 62

Sentiment distribution: Positive 11,807, Neutral 11,807, Negative 11,807 (perfectly balanced)

Star rating range: 1.05 to 5.0, mean 3.48, standard deviation 0.82

Tag count range: 0 to 19, mean 3.79

Help ratio range: 0.0 to 1.0, mean 0.15

1.4 Anonymization

Before any model training, the dataset undergoes anonymization to protect personally identifiable information. Student names are replaced with the token [STUDENT], roll numbers are replaced with [ROLL], email addresses are replaced with [EMAIL], and professor names are replaced with SHA-256 derived tokens prefixed with PROF_. This process ensures compliance with data privacy requirements while preserving the semantic content of the feedback text.

2. Baseline Model

2.1 Overview

The baseline model is a traditional machine learning approach that uses TF-IDF feature extraction combined with Logistic Regression for sentiment classification. It serves as the performance benchmark against which the more sophisticated models are evaluated. The model was developed in BASELINE_MODEL_1_with_annonimity.ipynb.

2.2 Libraries and Setup

The baseline model relies on the following Python libraries: pandas and openpyxl for data loading and manipulation, spacy with the en_core_web_sm model for named entity recognition during

anonymization, hashlib for generating SHA-256 based student identifiers, scikit-learn for TF-IDF vectorization, Logistic Regression training, and evaluation metrics, and matplotlib for confusion matrix visualization.

```
import pandas as pd
import re
import spacy
import hashlib
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
```

2.3 Data Loading and Anonymization

The raw dataset is loaded from an Excel file and passed through an anonymization pipeline. Student PII columns such as student_name, email, roll_no, and user_id are dropped from the dataframe. Each row is then assigned an anonymized student ID by hashing its index using SHA-256.

```
df = pd.read_excel("/content/edufeed_clean_data.xlsx")
student_pii = ['student_name', 'email', 'roll_no', 'user_id']
df.drop(columns=[c for c in student_pii if c in df.columns], inplace=True)


def generate_hash(text):
    return hashlib.sha256(text.encode()).hexdigest()


df['student_id'] = pd.Series(df.index.astype(str)).apply(generate_hash)
```

The comment text is further anonymized using spaCy NER to detect and replace PERSON entities with the token [STUDENT]. Regular expressions are applied to mask roll numbers matching the pattern of two letters followed by digits, and to replace email addresses with [EMAIL].

```
nlp = spacy.load("en_core_web_sm")


def mask_student_info(text):
    if pd.isna(text) or text == '':
        return ''
    text = str(text)
    doc = nlp(text)
    for ent in doc.ents:
        if ent.label_ == 'PERSON':
            text = text.replace(ent.text, '[STUDENT]')
    text = re.sub(r'\b\d{2}[A-Z]{2}\d{3,4}\b', '[ROLL]', text)
    text = re.sub(r'[\w\.-]+@[\w\.-]+\.\w+', '[EMAIL]', text)
    return text


df['comment_masked'] = df['comments'].apply(mask_student_info)
df.drop(columns=['comments'], inplace=True)
```

2.4 Label Generation

Sentiment labels are generated from the student_star rating column using a rule-based mapping. Ratings of 4 or above are classified as Positive, ratings of 3 are classified as Neutral, and ratings below 3 are classified as Negative.

```
def get_sentiment(star):
    if star >= 4:
        return 'Positive'
    elif star >= 3:
        return 'Neutral'
```

```
    else:
        return 'Negative'


df['sentiment'] = df['student_star'].apply(get_sentiment)
```

2.5 TF-IDF Vectorization

The masked comment text is used as the feature input. It is vectorized using TfidfVectorizer configured with a maximum of 5,000 features, unigrams and bigrams (ngram_range 1 to 2), and English stop word removal. The data is split into 80 percent training and 20 percent testing with stratified sampling to preserve class distribution.

```
X = df['comment_masked'].fillna('').astype(str)
y = df['sentiment']


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2),
    stop_words='english'
)
X_train_vec = tfidf.fit_transform(X_train)
X_test_vec  = tfidf.transform(X_test)
```

2.6 Model Training and Evaluation

Logistic Regression is trained on the TF-IDF features with a maximum of 1,000 iterations to ensure convergence. Model performance is measured using overall accuracy, a per-class classification

report covering precision, recall, and F1-score, and a confusion matrix visualized using ConfusionMatrixDisplay.

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train_vec, y_train)


y_pred = model.predict(X_test_vec)


print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))


cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix - Hashed + Anonymized Model")
plt.show()
```

2.7 Characteristics

The baseline model is lightweight and does not require a GPU. It is interpretable, fast to train, and serves as a strong starting point for sentiment classification. Its main limitations are that it treats comments as bags of words and does not capture semantic meaning or contextual relationships between words.

3. Advanced Multi-Layer Model

3.1 Overview

The advanced model, developed in MODEL.ipynb, is a multi-layer NLP pipeline that goes significantly beyond the baseline. It uses sentence embeddings, zero-shot classification, a FAISS vector index, cross-encoder reranking, and a generative language model to produce both sentiment

predictions and detailed professor insight reports. The pipeline is organized into six sequential layers.

3.2 Libraries and Setup

The model makes use of a broad set of libraries. The core NLP components include sentence-transformers with the all-MiniLM-L6-v2 model for semantic embedding generation and cross-encoder/nli-deberta-v3-base for zero-shot classification. FAISS is used for approximate nearest neighbor search. TinyLlama/TinyLlama-1.1B-Chat-v1.0 is used via Hugging Face transformers for insight generation. Additional utilities include spaCy, pandas, numpy, PyTorch, and hashlib.

```
import pandas as pd
import numpy as np
import spacy
import torch
import faiss
from sentence_transformers import SentenceTransformer
from sklearn.cluster import KMeans
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
from sentence_transformers import CrossEncoder
```

3.3 Layer 1: Data Ingestion

The first layer loads the raw Excel dataset using openpyxl, validates the schema against a required column list, coerces data types, drops rows missing professor_name, comments, or star_rating, removes duplicate records based on professor_name, comments, and post_date, and assigns a zero-padded review_id to each row. A structured JSON report is generated capturing row counts, column names, null value counts, and rating bound violations.

```
REQUIRED_COLUMNS = [
    "professor_name", "school_name", "department_name",
    "star_rating", "comments", "post_date",
```

```
    "student_star", "student_difficult",
    "sentiment_label", "tag_count", "course_mode",
]


df = load_raw(RAW_PATH)
```

```
df = coerce_types(df)
```

```
df, dropped_nulls = drop_nulls(df)
df, removed_dups = deduplicate(df)
df = add_row_id(df)
df.to_csv(OUT_CSV, index=False)
```

3.4 Layer 2: Anonymization

The second layer anonymizes professor names by computing a SHA-256 hash of each name and replacing it with a token of the form PROF_ followed by the first 8 characters of the hash in uppercase. All occurrences of professor name fragments in the comments field are replaced with the placeholder [PROF] using a compiled regular expression. The institution columns school_name and local_name are dropped from the dataset. A JSON mapping file is saved that records the original-to-token correspondence.

```
def sha256_token(v):
    return                                  "PROF_"                                  +
hashlib.sha256(v.lower().strip().encode()).hexdigest()[:8].upper()


def build_map(df):
```

```
    return {n: sha256_token(n) for n in df["professor_name"].dropna().unique()}


prof_map = build_map(df)
df["professor_token"] = df["professor_name"].map(prof_map)
```

```
df = df.drop(columns=["professor_name"])


regex = re.compile(r"\b(" + "|".join(map(re.escape, patterns)) + r")\b", re.I)
```

```
df["comments"] = comments.str.replace(regex, "[PROF]", regex=True)
```

3.5 Layer 3: Embedding and Inference

The third layer generates semantic embeddings using the SentenceTransformer model all-MiniLM-L6-v2. Embeddings are generated in batches of 64 with L2 normalization and saved as a NumPy array. Sentiment labels and emotion labels are predicted using zero-shot classification with the cross-encoder/nli-deberta-v3-base model. Sentiment classes are positive, negative, and neutral. Emotion classes are joy, anger, sadness, fear, surprise, and disgust. Processing is batched through a Hugging Face datasets pipeline for GPU efficiency.

```
SBERT_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
ZS_MODEL    = "cross-encoder/nli-deberta-v3-base"
SENTIMENT_LABELS = ["positive", "negative", "neutral"]
EMOTION_LABELS   = ["joy", "anger", "sadness", "fear", "surprise", "disgust"]

def get_embeddings(texts):
    model = SentenceTransformer(SBERT_MODEL, device="cuda")
    return model.encode(texts, batch_size=64,
                        show_progress_bar=True,
                        normalize_embeddings=True)


def dataset_zero_shot(texts, labels):
    clf = pipeline("zero-shot-classification", model=ZS_MODEL, device=device)
    ds = Dataset.from_dict({"text": texts})
    def infer(batch):
        out = clf(batch["text"], labels)
        return {"pred": [r["labels"][0] for r in out]}
    ds = ds.map(infer, batched=True, batch_size=32)
    return ds["pred"]


emb = get_embeddings(texts)
```

```
np.save(EMB_PATH, emb)
df["sentiment"] = dataset_zero_shot(texts, SENTIMENT_LABELS)
df["emotion"]   = dataset_zero_shot(texts, EMOTION_LABELS)
```

3.6 Layer 4: FAISS Index and Insight Generation

The fourth layer builds a FAISS HNSW (Hierarchical Navigable Small World) index over the sentence embeddings for fast approximate nearest neighbor retrieval. For each professor token, the mean embedding vector is computed across all their reviews, and the top 20 nearest neighbors are retrieved. A CrossEncoder model (cross-encoder/ms-marco-MiniLM-L-6-v2) reranks the candidate reviews to select the top 8 most relevant ones. These reviews are then passed to the TinyLlama-1.1B-Chat model to generate a structured insight card summarizing teaching characteristics, student sentiment, and improvement suggestions.

```
index = faiss.IndexHNSWFlat(dim, 32)
index.hnsw.efConstruction = 200
index.add(embeddings.astype("float32"))
faiss.write_index(index, INDEX_PATH)

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, device_map="auto",
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32
)

def generate_llm(prompt):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, max_new_tokens=180,
                             temperature=0.4, do_sample=True)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

3.7 Layer 5: QA Engine and RLHF Logging

The fifth layer provides a runtime question-answering interface. Given a query and professor token, a ContextBuilder retrieves the top relevant reviews and the professor's insight card. The QA engine constructs a structured prompt for TinyLlama and returns a concise answer. User feedback on generated answers is logged to a JSONL file for future reinforcement learning from human feedback (RLHF). The QAEngine also supports generating full professor insight reports on demand.

```
SYSTEM_PROMPT = "You are EduFeed AI. Answer only using context. Be concise."
```

```
class QAEngine:
    def ask(self, query, prof_token):
        reviews, card = self.ctx.build(query, prof_token)
        context = f"Professor: {prof_token}\n" \
                  f"Department: {card.get('department','Unknown')}\n\n" \
                  f"Reviews:\n{'---'.join(reviews)}"
        prompt = f"{SYSTEM_PROMPT}\n\nContext:\n{context}\n\nQuestion:\n{query}\n\nAnswer:"
        output = chat(prompt)[0]["generated_text"]
        return output.split("Answer:")[-1].strip()
```

3.8 Layer 6: Export and Reporting

The sixth layer collects all pipeline outputs and saves them for downstream use. The enriched predictions dataframe is exported as predictions.csv. The professor insight cards are copied as insights.json. A human-readable Markdown report is generated capturing the number of rows processed, sentiment distribution, and average star rating. All files are saved to an outputs directory.

```
def export_predictions():
```

```
    df = pd.read_csv(ENRICH_CSV, low_memory=False)
    df.to_csv(OUTPUTS / "predictions.csv", index=False)


def export_insights():
    shutil.copy2(INSIGHTS_JSON, OUTPUTS / "insights.json")
```

```

```

```
def export_report():
```

```
    text = f"# EduFeed Report\nGenerated: {datetime.now()}\n\n" \
           f"## Dataset\nRows: {len(df)}\n\n" \
           f"## Sentiment\n{sentiment}\n\n" \
           f"## Avg Rating\n{rating_avg}"
    out.write_text(text)
```

4. TensorFlow Deep Learning Model

4.1 Overview

The TensorFlow model, developed in tensorflow-model__5_.ipynb, is a deep learning approach that uses a Bidirectional LSTM network for sentiment classification. It incorporates a Groq API powered LLM relabelling step, a native TensorFlow TextVectorization layer for fast tokenization, and multi-GPU training using MirroredStrategy. This model targets the highest classification accuracy among the three approaches.

4.2 Libraries and Setup

The model uses TensorFlow and Keras for neural network construction and training. The Groq client library provides access to the llama-3.1-8b-instant model for LLM-based label refinement. scikit-learn is used for label encoding, train-test splitting, and evaluation metrics. seaborn and matplotlib are used for confusion matrix visualization. joblib is used for saving model artifacts.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from groq import Groq
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
import pandas as pd
```

4.3 Data Loading

The dataset is loaded from the balanced_edufeed_dataset.xlsx file. Only the comments and sentiment_label columns are retained. Rows with null values in either column are dropped. The comments column is cast to string and stripped of leading and trailing whitespace.

```
df = pd.read_excel(f'{data_path}/balanced_edufeed_dataset.xlsx')
df = df[['comments', 'sentiment_label']].dropna()
df['comments'] = df['comments'].astype(str).str.strip()

print(f'Total samples: {len(df)}')
print(df['sentiment_label'].value_counts())
```

4.4 LLM-Based Label Relabelling with Groq

A label refinement step is applied using the Groq API with the llama-3.1-8b-instant model. For each comment, a zero-temperature prompt asks the model to classify the sentiment as Positive, Negative, or Neutral using a single word response. Retry logic with exponential backoff handles API rate limits. A cache file is used to avoid redundant API calls across runs.

```
def groq_classify(text: str, retries: int = 3) -> str:
    prompt = (
        'Classify the sentiment of the following educational feedback comment.\n'
        'Reply with ONLY one word: Positive, Negative, or Neutral.\n\n'
        f'Comment: {text}\n\nSentiment:'
    )
    for attempt in range(retries):
        try:
            resp = client.chat.completions.create(
                model='llama-3.1-8b-instant',
                messages=[{'role': 'user', 'content': prompt}],
                max_tokens=5, temperature=0.0
            )
            raw = resp.choices[0].message.content.strip().lower()
            for word in re.split(r'\W+', raw):
                if word in VALID_LABELS:
                    return word.capitalize()
            return 'Neutral'
        except Exception as e:
            if attempt < retries - 1:
                time.sleep(2 ** attempt)
            else:
                return 'Neutral'
```

4.5 Anonymization

Before tokenization, comments are anonymized using regular expression patterns. Email addresses are replaced with the token EMAIL, Indian phone numbers are replaced with PHONE, URLs are replaced with URL, course and student ID codes are replaced with ID, large numbers are replaced with NUM, and capitalized proper name sequences are replaced with NAME.

```
PATTERNS = [
```

```
    (re.compile(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'), '<EMAIL>'),
    (re.compile(r'(\+91[\-\s]?)?[6-9]\d{9}'), '<PHONE>'),
    (re.compile(r'http\S+|www\.\S+'), '<URL>'),
    (re.compile(r'\b[A-Z]{2,4}\d{2,4}[A-Z]{0,3}\d{0,4}\b'), '<ID>'),
    (re.compile(r'\b\d{5,}\b'), '<NUM>'),
    (re.compile(r'(?<![.\n])\b[A-Z][a-z]{2,}(?:\s[A-Z][a-z]{2,})*\b'),
'<NAME>'),
]

def anonymize_text(text):
    if pd.isna(text):
        return ""
    for pattern, repl in PATTERNS:
        text = pattern.sub(repl, text)
    return text.strip()

df['comments'] = df['comments'].apply(anonymize_text)
```

4.6 Tokenization and Data Preparation

Labels are encoded using scikit-learn LabelEncoder. The data is split into training (70 percent), validation (15 percent), and test (15 percent) sets with stratified sampling. Text vectorization is performed using TensorFlow's TextVectorization layer configured with a vocabulary size of 20,000 tokens, a maximum sequence length of 100, and whitespace splitting for speed. The vectorizer is adapted on the first 5,000 training samples for efficiency.

```
label_encoder = LabelEncoder()
df['label_enc'] = label_encoder.fit_transform(df['sentiment_label'])
NUM_CLASSES = len(label_encoder.classes_)

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y)
```

```
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)


MAX_LEN = 100
VOCAB_SIZE = 20000
EMBEDDING_DIM = 64

vectorizer = tf.keras.layers.TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_sequence_length=MAX_LEN,
    split='whitespace'
)
vectorizer.adapt(X_train[:5000])
X_train_pad = vectorizer(X_train).numpy()
y_train_cat = keras.utils.to_categorical(y_train, NUM_CLASSES)
```

4.7 Model Architecture

The neural network is built inside a MirroredStrategy scope to enable multi-GPU training across both available T4 GPUs. The architecture consists of an Embedding layer with dimension 64, a SpatialDropout1D layer with a rate of 0.2 for regularization, two stacked Bidirectional LSTM layers with 64 and 32 units respectively each with a recurrent dropout of 0.2, a Dense layer with 64 units and ReLU activation, a Dropout layer with a rate of 0.4, and a final Dense output layer with softmax activation over the number of classes. The model is compiled with the Adam optimizer at a learning rate of 0.001 and categorical cross-entropy loss.

```
strategy = tf.distribute.MirroredStrategy()
with strategy.scope():
    inp = keras.Input(shape=(MAX_LEN,))
    x   = keras.layers.Embedding(VOCAB_SIZE, EMBEDDING_DIM)(inp)
    x   = keras.layers.SpatialDropout1D(0.2)(x)
    x   = keras.layers.Bidirectional(
```

```
            keras.layers.LSTM(64, return_sequences=True, dropout=0.2))(x)
    x   = keras.layers.Bidirectional(
            keras.layers.LSTM(32, dropout=0.2))(x)
    x   = keras.layers.Dense(64, activation='relu')(x)
    x   = keras.layers.Dropout(0.4)(x)
    out = keras.layers.Dense(NUM_CLASSES, activation='softmax')(x)

    model = keras.Model(inp, out)
    model.compile(
        optimizer=keras.optimizers.Adam(1e-3),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
```

4.8 Training

The model is trained for up to 20 epochs with a batch size of 128. Three callbacks are used: EarlyStopping monitors validation accuracy with a patience of 4 epochs and restores the best weights on termination, ModelCheckpoint saves the best model checkpoint to disk, and ReduceLROnPlateau halves the learning rate when validation loss plateaus for 2 consecutive epochs with a minimum learning rate of 1e-6.

```
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor='val_accuracy', patience=4, restore_best_weights=True),
    keras.callbacks.ModelCheckpoint(
        '/kaggle/working/best_model.keras',
        monitor='val_accuracy', save_best_only=True),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss', factor=0.5, patience=2, min_lr=1e-6)
]
```

```
history = model.fit(
    X_train_pad, y_train_cat,
    validation_data=(X_val_pad, y_val_cat),
    epochs=20,
    batch_size=128,
    callbacks=callbacks
)
```

4.9 Evaluation

After training, the best saved checkpoint is loaded and evaluated on the held-out test set. Test accuracy and test loss are reported. A full per-class classification report is printed with precision, recall, and F1-score for each of the three sentiment classes. A confusion matrix is computed and visualized as a heatmap using seaborn and saved as a PNG file.

```
best_model = keras.models.load_model('/kaggle/working/best_model.keras')
test_loss, test_acc = best_model.evaluate(X_test_pad, y_test_cat, verbose=0)
print(f'Test Accuracy : {test_acc:.4f}')
print(f'Test Loss     : {test_loss:.4f}')


y_pred = np.argmax(best_model.predict(X_test_pad), axis=1)
print(classification_report(y_test,                                          y_pred,
target_names=label_encoder.classes_))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.title('Confusion Matrix -- TF Model')
```

4.10 Model Artifact Saving

After training, the complete model is saved in Keras format. The LabelEncoder is serialized using joblib. The TextVectorization layer configuration and weights are saved separately as a JSON config file and a joblib pickle file respectively. These artifacts allow the full inference pipeline to be reconstructed without retraining.

```
joblib.dump(label_encoder, '/kaggle/working/label_encoder.pkl')


tokenizer_config = vectorizer.get_config()
tokenizer_weights = vectorizer.get_weights()


with open('/kaggle/working/tokenizer_config.json', 'w') as f:
    json.dump(tokenizer_config, f)


joblib.dump(tokenizer_weights, '/kaggle/working/tokenizer_weights.pkl')
```

4.11 Inference Pipeline

A standalone inference function allows new feedback to be analyzed at runtime. The user provides their name, email, roll number, professor name, a numeric rating, and free-text feedback. The personal identifiers are hashed and stored anonymously. The feedback text is run through the DistilBERT sentiment analysis pipeline (distilbert-base-uncased-finetuned-sst-2-english) and then through TinyLlama to generate a structured analysis covering sentiment interpretation, reason, key strength, key weakness, improvement suggestion, and student perception summary.

```
sentiment_model = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)
llm = pipeline(
    "text-generation",
    model="TinyLlama/TinyLlama-1.1B-Chat-v1.0"
)
```

```
def analyze_feedback():
    name     = input("Enter name: ")
    email    = input("Enter email: ")
    roll     = input("Enter roll no: ")
    professor = input("Enter professor name: ")
    rating   = int(input("Enter rating (1-5): "))
    feedback = input("Enter feedback: ")

    record = {
        "anon_id"  : hash(name + email + roll),
        "professor": professor,
        "rating"   : rating,
        "feedback" : feedback,
    }

    sentiment = sentiment_model(feedback[:512])[0]["label"]
    output                          =       llm(prompt,       max_new_tokens=200,
do_sample=False)[0]["generated_text"]
```